Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial: Vial

Instituto Tecnológico de Buenos Aires



Grupo G-139

Integrantes:

- Ramos, Máximo (64082)
- Rodriguez Falcon, Nicolás (63538)
- Procaccini, Juan Cruz (64571)

Fecha de Entrega: 15/11/2026

2026

Tabla de Contenidos

1 Introducción **2**

2 Modelo Computacional **2**

2.1 Dominio 2

2.2 Lenguaje 3

3 Implementación **5**

3.1 Frontend 5

3.2 Backend 5

3.3 Adicionales 8

3.4 Dificultades Encontradas 8

4 Futuras Extensiones **8**

5 Conclusiones **9**

6 Referencias **9**

7 Bibliografía **9**

1. Introducción

Vial es un lenguaje de dominio específico para describir redes viales urbanas y simular su comportamiento. Un programa Vial declara intersecciones, calles dirigidas, semáforos con fases, políticas que ajustan esas fases durante la simulación, rutas, flujos de vehículos y métricas definidas por el usuario. El compilador valida el programa y emite un único archivo HTML autocontenido que, abierto en un browser, simula y anima el escenario, muestra las métricas y permite exportarlas en formato JSON.

El compilador está escrito en C sobre el proyecto base Flex-Bison-Compiler [1], con Flex para el análisis léxico y Bison para el sintáctico. Este documento describe el dominio y el lenguaje (sección 2), la implementación de cada fase y las dificultades encontradas (sección 3), las extensiones posibles (sección 4) y las conclusiones (sección 5).

2. Modelo Computacional

2.1. Dominio

Planificar el tránsito implica evaluar intervenciones antes de llevarlas a cabo: cambiar el sentido de una calle, reprogramar un semáforo o cerrar un tramo. Vial permite describir un escenario de ese tipo como texto. Eso lo convierte en un artefacto de software, con lo que implica: se puede versionar, revisar entre pares, generar de forma paramétrica (por ejemplo, un corredor de n cuadras) y validar automáticamente en un pipeline de integración continua. Dos variantes de un escenario se comparan con un diff del código y de las métricas exportadas.

El usuario trabaja siempre en unidades del Sistema Internacional: duraciones en segundos, minutos u horas, distancias en metros o kilómetros y velocidades en km/h o m/s. La discretización es interna al compilador y al runtime: el tiempo avanza en pasos de 1 s y cada calle se divide en celdas de 7,5 m. Esos valores pueden cambiarse sin afectar al lenguaje.

El modelo de simulación es una red de flujo *localmente unidimensional*. Cada calle es una secuencia de celdas con un único carril, por donde los vehículos avanzan sin sobrepasarse, y las intersecciones son los puntos donde un vehículo pasa de una calle a la siguiente de su ruta. El modelo es deliberadamente simple:

- Una calle bidireccional se describe como dos calles opuestas.
- Cuando dos calles compiten por la misma calle de salida, gana la declarada primero. El orden de declaración también resuelve la prioridad entre flujos que ingresan por la misma calle.
- El ruteo es estático: cada ruta se resuelve una sola vez, en compilación.
- La emisión de vehículos es determinística, de modo que dos ejecuciones del mismo programa producen exactamente las mismas métricas.

El lenguaje no tiene iteración no acotada. Las políticas de los semáforos se evalúan una vez por paso de simulación, y la única estructura iterativa, `for`, tiene límites conocidos en compilación. Así, cada paso consume tiempo acotado y toda simulación termina, con un criterio análogo al que aplica el verificador de eBPF [2].

2.2. Lenguaje

Un programa empieza con el encabezado `simulation`, seguido de declaraciones en cualquier orden, con la única condición de que todo lo que se usa esté declarado antes. No hay terminadores de sentencia. Las palabras reservadas están en inglés, y los identificadores admiten letras acentuadas y ñ. El Código 1 muestra un escenario completo: un corredor sobre la avenida Córdoba con una transversal, un semáforo con una política adaptativa, dos flujos y tres métricas.

```
simulation Centro { duration 30 min }

intersection CórdobaPueyrredón at (0 m, 0 m)
intersection CórdobaCallao at 370 m east of CórdobaPueyrredón label "Córdoba y Callao"
intersection CórdobaMontevideo at 370 m east of CórdobaCallao
intersection CallaoRivadavia at 250 m south of CórdobaCallao

road CórdobaOeste from CórdobaPueyrredón to CórdobaCallao
road CórdobaEste from CórdobaCallao to CórdobaMontevideo limit 50 km/h
road Callao from CallaoRivadavia to CórdobaCallao length 260 m

duration VerdeBase = 30 s

policy Adaptativa(road principal, integer umbral) {
    if queue(principal) > umbral {
        extend principal by 10 s
        log "Extendida {principal} a los {elapsed}"
    } else {
        keep
    }
}

light Sem1 at CórdobaCallao {
    phase green for CórdobaOeste during VerdeBase
    phase green for Callao during VerdeBase - 5 s
    apply Adaptativa(CórdobaOeste, 15)
}

route Corredor = CórdobaPueyrredón -> CórdobaCallao -> CórdobaMontevideo
route Transversal = CallaoRivadavia -> CórdobaCallao -> CórdobaMontevideo

flow HoraPico along Corredor { spawn 2 every 10 s from 0 min to 15 min }
flow Continuo along Transversal { spawn 1 every 8 s }

metric Completados = count() on arrival
metric ViajePromedio = average(trip.travel_time) on arrival
metric PicoCórdobaOeste = maximum(occupancy(CórdobaOeste)) on tick
```

Código 1: Escenario completo en Vial.

Las construcciones del lenguaje son las siguientes:

Intersecciones
Toda intersección tiene posición. La posición puede ser absoluta, en un plano cartesiano con el eje Y hacia arriba (`at (0 m, 0 m)`), o relativa a otra intersección (`at 370 m east of X`). La preposición `at` denota siempre un posicionamiento relativo: de una intersección respecto de un punto o de otra intersección, y de un semáforo respecto de la intersección que regula.

Calles
Son dirigidas. Su largo es opcional y por defecto vale la distancia entre sus extremos; se

declara cuando la calle no es recta. El límite de velocidad también es opcional y por defecto vale 40 km/h.

Semáforos
Tienen una secuencia de fases que se alternan en orden. Cada fase habilita una o más calles que entran a su intersección durante una duración.

Políticas
Son procedimientos parametrizados que se aplican a un semáforo con `apply` y se evalúan una vez por paso. Pueden consultar el estado con `queue` (vehículos detenidos frente al semáforo), `occupancy` (porcentaje de celdas ocupadas) y `elapsed` (tiempo transcurrido), y actuar con `extend` (prolongar la fase vigente si habilita la calle indicada), `keep` y `log`.

Rutas y flujos
Una ruta es una secuencia de intersecciones. Un flujo emite vehículos sobre una ruta con una tasa (`spawn n every d`) y, opcionalmente, una ventana temporal (`from a to b`). Los vehículos que no pueden ingresar esperan fuera de la red.

Métricas
Las define el usuario como una agregación (`count`, `sum`, `average`, `minimum` o `maximum`) de una expresión evaluada en cada evento: `tick`, al final de cada paso, o `arrival`, cada vez que un vehículo completa su ruta, con acceso a `trip.travel_time`. Agregar una métrica nueva es escribir una línea en el programa.

Constantes, arreglos e iteración
Las constantes tienen tipo explícito y se resuelven en compilación. Los arreglos se definen por declaraciones indexadas, en general dentro de un `for`, como en el Código 2. El rango del `for` es inclusivo y se conoce en compilación, de modo que el compilador aplica *loop unrolling* a cada `for`, como una optimización interna: el programa resultante equivale a escribir cada declaración por separado.

Strings
Admiten interpolación de expresiones con `{expr}`, tanto en los `log` como en los `label`. Las magnitudes se muestran con su unidad (`1 min 30 s`, `50 km/h`) y las entidades, por su etiqueta o su nombre.

```
intersection Rivadavia[0] at (0 m, 0 m)
for i in 1 .. 4 {
    intersection Rivadavia[i] at 100 m east of Rivadavia[i - 1] label "Rivadavia {i}"
    road TramoRivadavia[i] from Rivadavia[i - 1] to Rivadavia[i] limit 40 km/h
}
```

Código 2: Generación de un corredor de cinco intersecciones mediante iteración acotada.

El sistema de tipos es estático. Además de `integer`, `boolean` y `string`, hay tres tipos con dimensión, `duration`, `distance` y `speed`, y los tipos de las entidades (`intersection`, `road`, `light`, `route`, `flow`), que se usan como parámetros de las políticas. La Tabla 1 resume las reglas de las operaciones.

Respecto de la especificación del Stage I, el lenguaje incorporó la devolución de la cátedra:

- Unidades del Sistema Internacional en lugar de ticks y celdas, y el valor `elapsed` en lugar de `tick`.
- Límite de velocidad por calle.
- Posiciones obligatorias en metros, en un plano cartesiano, absolutas o relativas.

Operación	Operandos válidos	Resultado
`+ -`	dos enteros, o dos magnitudes de la misma dimensión	el mismo tipo
`*`	dos enteros, o un entero y una magnitud	entero o magnitud
`/`	dos enteros, o una magnitud y un entero (trunca hacia cero)	entero o magnitud
`< <= > >=`	dos enteros, o dos magnitudes de la misma dimensión	`boolean`
`= !=`	dos valores del mismo tipo, incluidas las entidades	`boolean`
`and or not`	booleanos	`boolean`

Tabla 1: Reglas de tipos. Un literal con unidad (`30 s`, `2 km`, `60 km/h`) determina la dimensión.

- Sentencias sin `;`.
- Interpolación nombrada en los strings.
- Métricas definidas por el usuario en lugar de un bloque `report` con métricas fijas.
- Identificadores con letras acentuadas.

3. Implementación

El compilador lee el programa por entrada estándar y lo procesa en el siguiente pipeline:

Flex → Bison → AST → análisis semántico → modelo → generador → HTML

El HTML se escribe en la salida estándar y los mensajes de diagnóstico, en la salida de error. Si el programa se rechaza, el código de salida es distinto de cero y no se emite nada.

3.1. Frontend

El analizador léxico opera en el modo *push* del proyecto base: cada acción emite cero o más tokens hacia el parser. Las unidades (`m`, `km`, `s`, `min`, `h`, `km/h`, `m/s`) son palabras reservadas. Los operadores `≤ ≥ ≠` se aceptan junto con `<= >= !=`, y los identificadores admiten las letras latinas acentuadas de UTF-8.

La interpolación de strings se resuelve con una pila de contextos de Flex: un contexto exclusivo para el texto del string y uno inclusivo para la expresión interpolada. Así, el parser recibe la expresión como tokens normales y la analiza con la misma gramática. Los errores léxicos indican la línea y, en los casos frecuentes, una pista; por ejemplo, un `;` se reporta aclarando que el lenguaje no usa terminadores.

La gramática de Bison no tiene conflictos. La precedencia de los operadores se declara con `%left`, `%right` y `%nonassoc`, y las ubicaciones (`%locations`) registran en los nodos del AST la línea de origen, que después usan los mensajes de error semánticos. El AST tiene un tipo de nodo por construcción, con destructores recursivos. Todo el compilador se construye con AddressSanitizer, de modo que una pérdida de memoria en un programa válido hace fallar las pruebas.

3.2. Backend

Análisis semántico. El análisis recorre las declaraciones en orden, en una sola pasada, y construye un *modelo* con todas las entidades ya resueltas.

- **Tabla de símbolos.** Es una tabla de hash con una pila de scopes: el global, el de cada `for` (con la variable reasignada en cada iteración) y el de los parámetros de cada política. No se permite que un nombre oculte a otro visible, y los nombres predefinidos (`queue`, `occupancy`, `elapsed`, `trip`, los agregadores y los eventos) están reservados. Los elementos de un arreglo se registran con su índice resuelto, como `Rivadavia[3]`.
- **Unrolling y plegado de constantes.** Cada `for` se desenrolla y su cuerpo se analiza una vez por valor de la variable. Las expresiones que se conocen en compilación se pliegan a su valor. Las magnitudes se representan como enteros exactos en unidades base (metros, segundos y metros por hora), así que no hay aritmética de punto flotante, y el desborde y la división por cero se detectan como errores.
- **Tipos y contextos.** Cada expresión queda anotada con su tipo, que es un atributo sintetizado desde las hojas. Además, se controla el contexto: las consultas al estado de la simulación solo pueden aparecer en las políticas y en las métricas, y `trip`, solo en las métricas `on arrival`.
- **Validaciones del dominio.** Se reportan como `Semantic error at line N: …` (por ejemplo, `road "AD" references undeclared intersection "D"`). Son las siguientes:
  - identificadores duplicados y referencias a entidades o índices no declarados, o de una clase distinta de la esperada;
  - tipos y unidades incompatibles;
  - longitudes, duraciones, velocidades y tasas no positivas, y calles con origen igual al destino;
  - intersecciones en la misma posición;
  - tramos de ruta sin calle, o con más de una calle en el mismo sentido;
  - ventanas de flujo que terminan antes de empezar;
  - fases sobre calles que no entran a la intersección del semáforo, y calles entrantes que no figuran en ninguna fase;
  - más de un semáforo por intersección;
  - aplicación de políticas con una cantidad o un tipo de argumentos incorrectos;
  - funciones, miembros, agregadores y eventos desconocidos.

Se reportan todos los errores del programa, no solo el primero, y una expresión con error no genera mensajes en cascada.

Generación de código. El generador recorre el modelo y produce un HTML con tres partes:

1. Los datos del escenario, como un objeto JSON: intersecciones, calles con su cantidad de celdas, semáforos, rutas ya resueltas en calles, flujos y métricas.
2. La lógica del programa, como funciones JavaScript: una por política y una por cada métrica que evalúa una expresión. Se traducen nodo por nodo, con un procedimiento de generación por tipo de nodo del AST [3]: las constantes se reemplazan por su valor plegado, los parámetros por argumentos posicionales y las consultas por llamadas a una interfaz mínima del runtime.
3. El runtime, que es fijo e igual para todos los programas.

Runtime. Siguiendo el criterio de [4], el compilador incorpora el runtime completo en la salida. El HTML resultante solo necesita un browser y no depende de ningún servidor ni de bibliotecas externas.

El runtime es un componente versionado en el repositorio e independiente de cada programa. Tiene un motor de simulación determinístico, sin acceso a la interfaz, y un visor (Figura 1). El visor dibuja la red en SVG, invirtiendo el eje Y del plano cartesiano, y muestra:

- las celdas libres, ocupadas y detenidas;
- las fases de cada semáforo con su tiempo restante;
- las métricas del programa y la ocupación de cada calle;
- una traza de eventos que incluye los `log` de las políticas.

La velocidad de reproducción es un multiplicador del tiempo simulado (×1 a ×60), y el botón *Exportar JSON* descarga las métricas.

Esta separación atiende la observación de la cátedra sobre mezclar código generado con la interfaz del prototipo presentado en el Stage I: el prototipo quedó solo como referencia visual, el runtime se construyó sobre un contrato de datos acotado y el generador no emite código de interfaz.

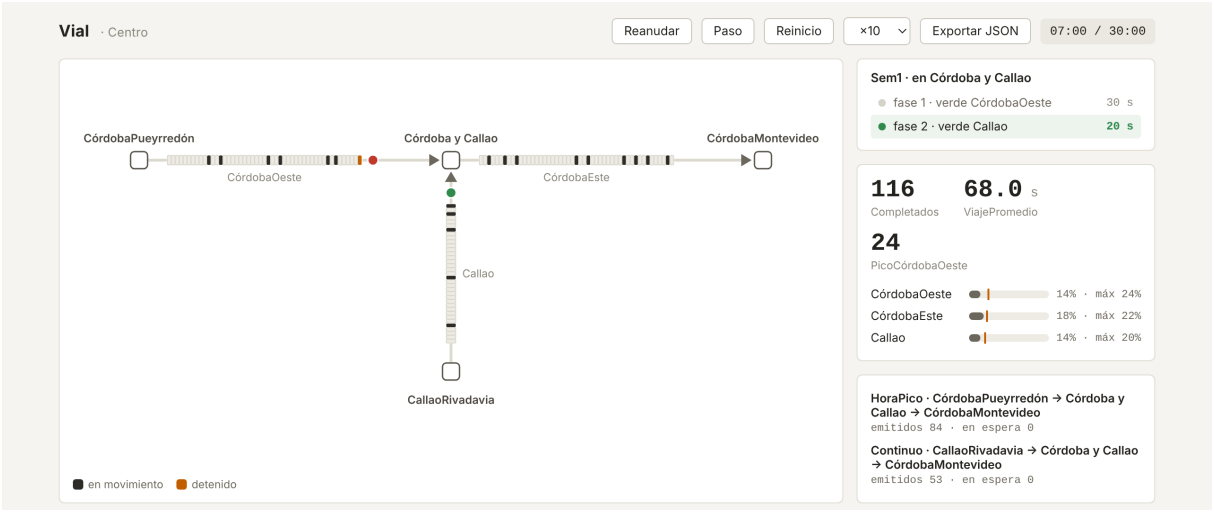


Figura 1: Visor generado para el escenario del Código 1, a los 7 minutos de simulación.

En cada paso de simulación, el motor:

1. emite los vehículos de cada flujo;
2. evalúa las políticas;
3. mueve los vehículos;
4. ingresa a la red los que esperan;
5. avanza los semáforos;
6. evalúa las métricas `on tick`.

Las decisiones de movimiento se toman sobre el estado del comienzo del paso.

Cada vehículo acumula, en cada paso, la distancia que permite el límite de su calle, y avanza tantas celdas como completa ese acumulador, mientras estén libres. Así, el límite de velocidad se respeta en promedio con aritmética entera. Por ejemplo, una calle de 75 m a 27 km/h se recorre en 11 s, y a 60 km/h, en 6 s.

Pruebas. El script de pruebas del proyecto base ejecuta 30 programas que deben aceptarse y 40 que deben rechazarse: 15 por errores léxicos o sintácticos y 25 por errores semánticos. Cada programa de rechazo contiene un único defecto. Las pruebas corren en integración continua en cada push.

3.3. Adicionales

Además de lo pedido en la consigna, el compilador acepta comentarios de bloque, identificadores con letras acentuadas y los operadores `≤ ≥ ≠`, y sus mensajes de error indican la línea y la entidad involucrada.

3.4. Dificultades Encontradas

- **Gramática sin terminadores.** Quitar el `;` exige que ninguna palabra reservada pueda continuar una expresión. Por eso la ventana de un flujo pasó de `between a and b` a `from a to b`, ya que `and` es el operador lógico.
- **Unidades sin punto flotante.** Las velocidades en km/h no tienen una conversión entera exacta a m/s. Se resolvió representando las velocidades en metros por hora, lo que mantiene toda la aritmética en enteros y permite que el compilador, en C, y el runtime, en JavaScript, produzcan exactamente los mismos valores y el mismo texto al formatear magnitudes.
- **Discretización y límites de velocidad.** Con pasos de 1 s y celdas de 7,5 m, avanzar una celda por paso equivale a una velocidad fija de 27 km/h y vuelve irrelevante el límite de cada calle. Se resolvió con el acumulador de avance descripto en la sección anterior.
- **Métricas dentro de un `for`.** Al desenrollar el bucle, todas las iteraciones comparten los mismos nodos del AST, pero una referencia como `T[i]` resuelve a una calle distinta en cada una. La resolución de cada referencia se guarda por iteración, y cada métrica recuerda a qué iteración corresponde.
- **Datos del usuario dentro del HTML.** Las etiquetas pueden contener cualquier texto, incluso `</script>`. Todos los strings se escapan en un único lugar del generador antes de incorporarse al JSON o al código.

4. Futuras Extensiones

- Calles con varios carriles y sobrepaso. Requiere extender la sintaxis de las calles y el modelo de movimiento.
- Ruteo dinámico, en el que los vehículos eligen su camino según la congestión.
- Emisión estocástica de vehículos con una semilla explícita, para conservar la reproducibilidad.
- Más información sobre cada viaje en las métricas (por ejemplo, el tiempo de espera fuera de la red) y nuevos eventos.
- Una ejecución sin interfaz, desde la línea de comandos, que produzca el JSON de métricas directamente, para comparar escenarios en un pipeline.

5. Conclusiones

Se implementó un compilador completo para Vial. Valida estáticamente la estructura de la red, los tipos y las unidades, y genera un artefacto autocontenido que simula el escenario de forma determinística. La separación entre lo que genera el compilador (datos y lógica) y un runtime fijo mantuvo acotado el generador y permitió verificar el motor de simulación de forma independiente. Las correcciones del Stage I, en particular el uso de unidades del Sistema Internacional y las métricas definidas por el usuario, acercaron el lenguaje al dominio: el usuario describe calles en metros y simulaciones en minutos, y la discretización queda como un detalle interno.

6. Referencias

[1] Alpha-Theta-Gamma-Mu. *Flex-Bison-Compiler*, rama `development`. `https://github.com/Alpha-Theta-Gamma-Mu/Flex-Bison-Compiler`

[2] The Linux Kernel Documentation. *eBPF verifier*. `https://docs.kernel.org/bpf/verifier.html`

[3] Cátedra de Autómatas, Teoría de Lenguajes y Compiladores, ITBA (2024-05-16). *Generación de Código*, v0.1.0.

[4] Cátedra de Autómatas, Teoría de Lenguajes y Compiladores, ITBA (2024-05-16). *Runtime*, v0.1.0.

7. Bibliografía

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.ª ed.). Addison-Wesley.

Cátedra de Autómatas, Teoría de Lenguajes y Compiladores, ITBA (2026-08-06). *Proyecto Especial: Diseño e Implementación de un Lenguaje*.

Cátedra de Autómatas, Teoría de Lenguajes y Compiladores, ITBA (2026-03-12). *Modelo Computacional*.

Cátedra de Autómatas, Teoría de Lenguajes y Compiladores, ITBA (2025-09-03). *Análisis Léxico*, v1.0.0.

Cátedra de Autómatas, Teoría de Lenguajes y Compiladores, ITBA (2024-05-08). *Análisis Sintáctico*, v0.1.0.

Cátedra de Autómatas, Teoría de Lenguajes y Compiladores, ITBA (2024-09-25). *Análisis Semántico*, v0.2.0.

Free Software Foundation. *GNU Bison Manual*. `https://www.gnu.org/software/bison/manual/`

Paxson, V. et al. *Lexical Analysis With Flex*. `https://westes.github.io/flex/manual/`